

Chapter 02

Core Components of LangChain

A guided tour of the six building blocks — Models, Prompts, Chains, Indexes, Memory, and Agents — that every LangChain application is assembled from.

WHAT THIS CHAPTER COVERS

- ▶ The six LangChain components, at a glance
- ▶ Models: standardizing access to any LLM
- ▶ Language models vs. embedding models
- ▶ Prompts: dynamic, role-based & few-shot
- ▶ Chains: sequential, parallel & conditional
- ▶ Indexes: connecting to external knowledge
- ▶ Memory: solving the statelessness problem
- ▶ Agents: reasoning + tools = taking action

Core Components of LangChain

RECAP — CHAPTER 1

LangChain is an open-source framework whose core value is **orchestration**: it connects document loaders, text splitters, embedding models, vector databases, and LLMs into a single pipeline. Its key benefits are **chains** (automatic wiring between components), **model-agnostic development** (swap providers in a couple of lines), a complete ecosystem of integrations, and built-in memory/state handling. This chapter opens up each of those ideas as a concrete, named component.

1. The Six Building Blocks of LangChain

Every concept in LangChain can be organized under six components. Studying these six in depth covers the large majority of what the framework offers — and every future chapter in this roadmap is simply a deeper dive into one of these six.

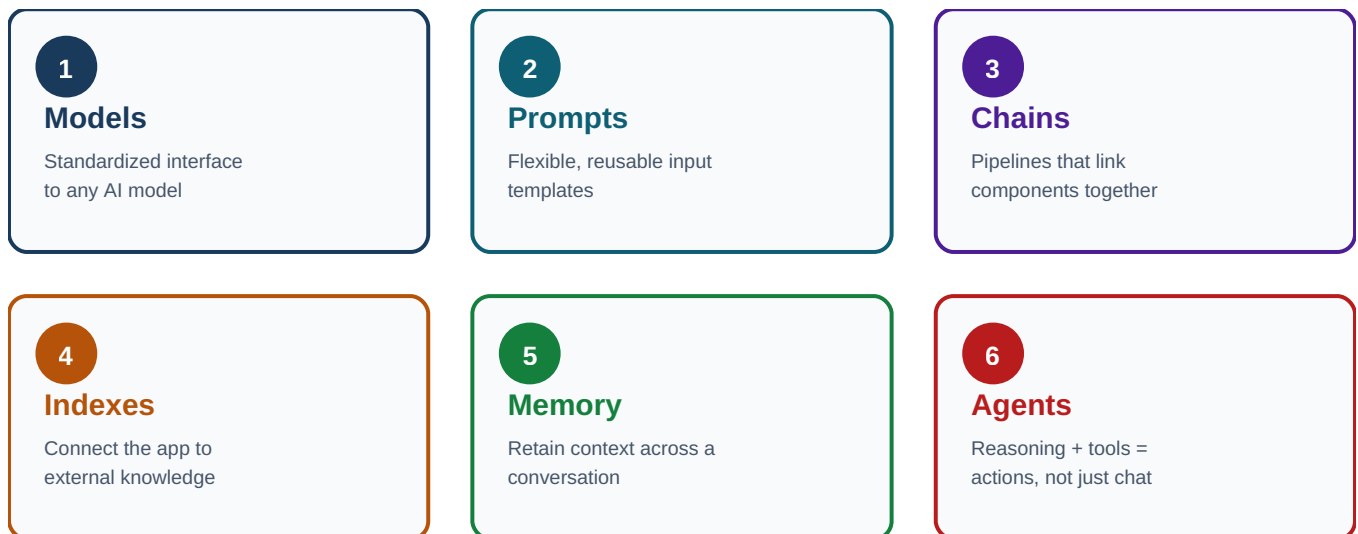


Figure 2.1 — The six components of LangChain.

#	Component	What It Is Responsible For
1	Models	The core interface through which you interact with AI models (language models and embedding models).
2	Prompts	The inputs provided to an LLM — made dynamic, reusable, role-based, or few-shot.
3	Chains	Pipelines that connect components so one step's output becomes the next step's input automatically.
4	Indexes	The mechanism that connects an application to external knowledge (PDFs, websites, databases).
5	Memory	Gives an otherwise stateless LLM API the ability to remember earlier parts of a conversation.
6	Agents	Chatbots with reasoning ability and tool access, capable of taking actions, not just replying.

Quick Revision — Six Components

- Models, Prompts, Chains, Indexes, Memory, Agents.
- These six components map directly onto the rest of this course — each gets its own deep-dive chapter.
- Together, they cover interacting with AI models, shaping inputs, building pipelines, using external knowledge, retaining context, and taking action.

2. Component 1: Models

Definition

In LangChain, **models are the core interface through which you interact with AI models.**

Why It Exists — A Short History

Across the history of NLP, the chatbot was the application everyone wanted to build, but building one required solving two hard problems: **Natural Language Understanding (NLU)** — correctly understanding what a user means — and **context-aware text generation** — replying appropriately once the query is understood. LLMs, trained on enormous amounts of internet data, solved both problems together. This created a new problem: because LLMs are trained on so much data, they contain billions of parameters and are often well over 100 GB in size — far too large to run on a personal computer or even a small company's own servers. This was solved by **LLM APIs** from providers such as OpenAI and Anthropic: instead of hosting the model yourself, you call an API and pay only for what you use.

The Problem Models Actually Solves: Standardization

Solving the hosting problem with APIs created a *third* problem: different LLM providers implemented their APIs differently. An application that needs to talk to two different LLM providers ends up needing two different pieces of integration code — and switching providers later (for example, moving off a costly API) means rewriting that integration code.

RAW OPENAI API

```
# Native OpenAI SDK
from openai import OpenAI
client = OpenAI()

completion = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system",
         "content": "You are a helpful
assistant."},
        {"role": "user",
         "content": "Write a haiku about
recursion."}
    ]
)

print(completion.choices[0].message)
```

RAW ANTHROPIC API

```
# Native Anthropic SDK
import anthropic
client = anthropic.Anthropic()

message = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1000,
    temperature=0,
    system="You are a world-class poet. "
        "Respond only with short poems.",
    messages=[
        {"role": "user",
         "content": [{"type": "text",
                        "text": "Why is the ocean
salty?"}]}
    ]
)

print(message.content)
```

Notice that the two SDKs above are structured quite differently — different client objects, different call signatures, different response shapes. Now compare the same two providers accessed through LangChain:

LANGCHAIN + OPENAI

```
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(model='gpt-4', temperature=0)

result = model.invoke("Now divide the result by
1.5")

print(result.content)
```

LANGCHAIN + ANTHROPIC

```
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv

load_dotenv()

model = ChatAnthropic(model='claude-3-
opus-20240229')

result = model.invoke("Hi who are you")

print(result.content)
```

Notice that the two SDKs above are structured quite differently — different client objects, different call signatures, different response shapes. Now compare the same two providers accessed through LangChain above.

IMPORTANT

Switching providers in LangChain is, in practice, a change of one import line and one constructor call — `ChatOpenAI(...)` becomes `ChatAnthropic(...)`. The `.invoke()` call and the way the result is read (`result.content`) stay identical.

Two Kinds of Models

LangChain lets you communicate with two categories of AI models:

Model Type	Input → Output	Primary Use
Language Models (LLMs)	Text in → Text out	Chatbots, AI agents, and any application built on conversational text generation.
Embedding Models	Text in → Vector out	Semantic search — representing meaning as a vector for similarity comparison.

LangChain's documentation lists chat-model integrations for providers such as Anthropic, Mistral AI, Azure, OpenAI, Vertex AI, AWS Bedrock, and Hugging Face — along with which features each one supports (tool calling, structured/JSON output, local execution, multimodal input). A similar page lists supported embedding-model providers, including OpenAI, Mistral AI, IBM, and Llama-based models.

INTERVIEW TIP

Be ready to name the two model categories LangChain supports (language models and embedding models) and explain the input/output shape of each — this is a very common warm-up question.

COMMON MISTAKE

Assuming the Models component makes every provider behave *identically* in every respect. It standardizes the calling interface and response shape, but provider-specific features (e.g., certain tool-calling formats or structured-output modes) can still vary.

BEST PRACTICE

Design new LangChain applications against the standardized model interface from day one, even if you only plan to use a single provider today — it keeps a future provider switch to a couple of lines of code.

🔄 Quick Revision — Models

- Models = the standardized interface LangChain gives you to talk to any AI model.
- It exists to solve implementation inconsistency across providers' native APIs.
- Switching providers in LangChain is typically a 1–2 line code change.
- Two categories: Language models (text → text) and Embedding models (text → vector).

3. Component 2: Prompts

Definition

Prompts are the inputs provided to an LLM. If you type “What is LangChain?” into ChatGPT, that string is the prompt.

Why Prompts Matter

An LLM's output is extremely sensitive to its prompt. Asking the same underlying question with a single word changed — for example, “*Explain linear regression in an academic tone*” versus “*Explain linear regression in a fun tone*” — can produce very different outputs. This sensitivity is significant enough that an entire discipline, **prompt engineering**, and a job title, **prompt engineer**, have emerged around it in the last two years.

REAL WORLD INSIGHT

Prompt engineering has attracted some mockery on social media, but the underlying skill — reliably shaping LLM behavior through prompt design — is a genuine and valuable area of expertise.

1. Dynamic & Reusable Prompts

Because the exact topic or tone a user wants is often unknown in advance, LangChain lets you build a prompt template with placeholders that get filled in at call time.

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
    'Summarize {topic} in {emotion} tone')

print(prompt.format(topic='Cricket', emotion='fun'))
```

The same template can be reused for a completely different request — for example, summarizing “Biology” in a “serious” tone — without writing a new prompt string each time.

2. Role-Based Prompts

LangChain also supports structuring prompts as a sequence of roles — typically a **system** message that sets behavior, and a **user** message that carries the actual request.

```
# Define the ChatPromptTemplate using from_template
chat_prompt = ChatPromptTemplate.from_template([
    ("system", "Hi you are a experienced {profession}"),
    ("user", "Tell me about {topic}"),
])

# Format the prompt with the variables
formatted_messages = chat_prompt.format_messages(
    profession="Doctor", topic="Viral Fever")
```

Filling `profession="Doctor"` and `topic="Viral Fever"` effectively guides the LLM to answer *as an experienced doctor* discussing viral fever. A different user could reuse the same template with `profession="Engineer"` and `topic="Developing Bridges"`.

3. Few-Shot Prompting

In **few-shot prompting**, the LLM is first shown a handful of worked examples, and then asked to classify or respond to a new case using the same pattern. This is especially useful for classification-style tasks, such as a customer support ticket classifier.

```
examples = [
    {"input": "I was charged twice for my subscription this month.", "output": "Billing Issue"},
    {"input": "The app crashes every time I try to log in.", "output": "Technical Problem"},
    {"input": "Can you explain how to upgrade my plan?", "output": "General Inquiry"},
    {"input": "I need a refund for a payment I didn't authorize.", "output": "Billing Issue"},
]

# Step 2: Create an example template
example_template = """
Ticket: {input}
Category: {output}
"""

# Step 3: Build the few-shot prompt template
few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=PromptTemplate(input_variables=["input", "output"],
    template=example_template),
    prefix="Classify the following customer support tickets into one of the "
    "categories: 'Billing Issue', 'Technical Problem', or 'General Inquiry'.\n\n",
    suffix="\nTicket: {user_input}\nCategory:",
    input_variables=["user_input"],
)
```

When a fifth, unseen ticket — “*I am unable to connect to the internet using your service.*” — is passed in as `user_input`, the LLM classifies it correctly as a **Technical Problem**, having learned the pattern from the four preceding examples.

Prompt Style	What It Solves
Dynamic & reusable	Avoids hardcoding a new prompt string for every possible topic/tone combination.
Role-based	Lets you assign the LLM a persona (e.g., "experienced doctor") for more targeted responses.
Few-shot	Teaches the LLM a task pattern using worked examples, useful for classification-style tasks.

COMMON MISTAKE

Changing wording in a prompt template casually and assuming the output will be unaffected. Because LLM output is highly prompt-sensitive, even small wording changes deserve re-testing.

🔄 Quick Revision — Prompts

- Prompts = the input given to an LLM; output is highly sensitive to prompt wording.
- Dynamic/reusable prompts use placeholders filled in with `.format(...)`.
- Role-based prompts assign a persona via a system message, then ask via a user message.
- Few-shot prompts show worked examples before asking the LLM to handle a new case.

4. Component 3: Chains

Definition

Chains are the component that lets you build pipelines in LangChain — in fact, the framework is named after this concept.

Why It Exists

Consider an application that takes a large English text as input and must return a Hindi summary under 100 words. The natural flow is: send the input to one LLM to translate it into Hindi, then send that translated text to a second LLM to summarize it. Built manually, every stage's output has to be captured and re-supplied as the next stage's input by hand. A **chain** automates exactly that hand-off: the output of one stage automatically becomes the input of the next, with no manual wiring code required.

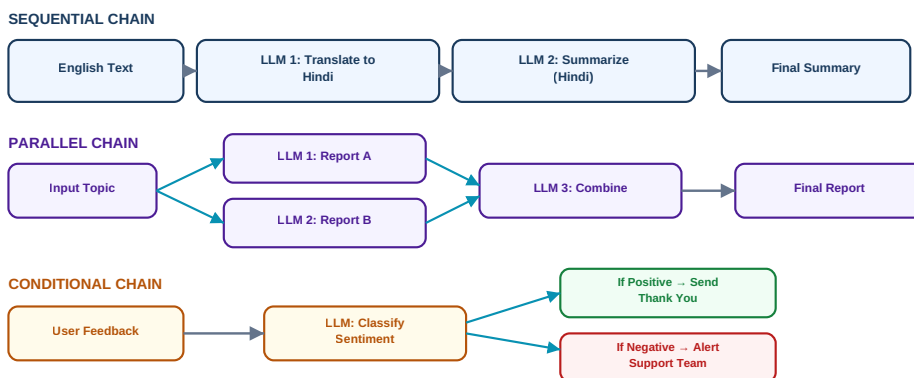


Figure 2.2 — Three chain patterns: sequential, parallel, and conditional.

Chain Type	Description	Example
Sequential	Stages run one after another; each stage's output feeds the next.	Translate English → Hindi, then summarize the Hindi text.
Parallel	Multiple stages run at the same time on the same input, then their outputs are combined.	Two LLMs each generate a report on an incident; a third LLM merges both reports.
Conditional	Different processing paths are taken depending on a condition.	Positive feedback → auto-reply "thank you"; negative feedback → alert the support team.

BEST PRACTICE

Design the shape of your pipeline (sequential vs. parallel vs. conditional) before writing any chain code — the structure of the problem should decide the chain type, not the other way around.

🔄 Quick Revision — Chains

- Chains build pipelines; one stage's output automatically becomes the next stage's input.
- Sequential chains: linear, one stage after another.
- Parallel chains: multiple stages run simultaneously, then results are combined.
- Conditional chains: branch processing based on a condition.

5. Component 4: Indexes

Definition

Indexes connect your application to external knowledge, such as PDFs, websites, and databases.

Why It Exists

A general-purpose model like ChatGPT can answer most questions because it was trained on a huge slice of the internet. But it cannot answer questions about knowledge it never saw during training — for example, “*What is the leave policy of my company XYZ?*” or “*What is the notice period policy of my company XYZ?*” The solution is to connect the LLM to an **external knowledge source** — such as the company's own rulebook — so it can answer questions grounded in that private data, while still answering general questions (like who the current Prime Minister is) from its own training.

The Four Sub-Components of Indexes

Sub-Component	Responsibility
Document Loader	Brings data in from wherever it is stored (cloud storage, local files, PDFs, websites, databases).
Text Splitter	Breaks the loaded document into smaller chunks (by page, paragraph, or chapter).
Vector Store	Stores the embedding vector generated for each chunk, so it can be queried later.
Retriever	Embeds an incoming query, runs a semantic search against the vector store, and returns the most relevant chunks.

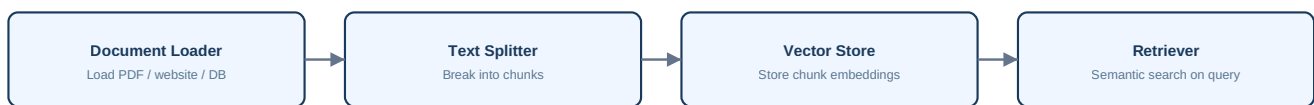


Figure 2.3 — The four sub-components of Indexes, in pipeline order.

Together, these four sub-components let an LLM application answer questions grounded in a specific, private knowledge source — the same underlying pipeline discussed in Chapter 1's document-chat case study.

REAL WORLD INSIGHT

The external knowledge source does not have to be a document at all — it can equally be a live website or a company database, which is why LangChain keeps this component flexible rather than PDF-specific.

Quick Revision — Indexes

- Indexes connect an LLM application to external knowledge (PDFs, websites, databases).
- Four sub-components: Document Loader → Text Splitter → Vector Store → Retriever.
- Solves the problem of an LLM having no knowledge of private/proprietary data.

6. Component 5: Memory

Definition & The Problem It Solves

LLM API calls are stateless — each call is independent and carries no memory of any previous call. For example, asking an LLM “Who is Narendra Modi?” gets a correct answer, but a follow-up call asking simply “How old is he?” fails, because the API has no memory of the earlier question — the model has no idea who “he” refers to. For any conversational application, this is a serious usability problem: without memory, a user would have to restate context on every single turn.

Types of Memory in LangChain

Memory Type	How It Works	Trade-off
Conversation Buffer Memory	Stores the entire chat history and resends all of it with every new API call.	Simple, but grows expensive as the conversation gets long.
Conversation Buffer Window Memory	Stores only the last N interactions (e.g., the last 100 messages) and continuously updates this window.	Bounded cost, but older context beyond the window is lost.
Summarizer-Based Memory	Maintains a running summary of the conversation so far, and sends the summary instead of the full history.	Saves tokens/cost, at the risk of losing fine detail.
Custom Memory	Stores specialized pieces of information for advanced use cases (e.g., specific user preferences or facts).	Tailored to the application, but requires custom design.

COMMON MISTAKE

Building a multi-turn assistant and forgetting to attach any memory component, then being surprised that the model cannot resolve follow-up references like “he” or “the second one.”

🔄 Quick Revision — Memory

- LLM API calls are stateless by default — no memory of prior calls.
- Conversation Buffer Memory: stores and resends the entire chat history.
- Conversation Buffer Window Memory: stores only the last N interactions.
- Summarizer-based Memory: sends a running summary instead of full history.
- Custom Memory: stores specialized facts/preferences for advanced use cases.

7. Component 6: Agents

Definition

An **AI agent** is a chatbot with **superpowers**: instead of only replying in conversation, it can also take actions on the user's behalf.

Agents vs. Chatbots

Aspect	Chatbot	AI Agent
Can hold a conversation	Yes	Yes
Has reasoning capability	No	Yes
Has access to tools	No	Yes
Can take real-world actions	No	Yes (e.g., call an API, book a ticket)

For example, a travel-site chatbot can tell you that hill stations like Shimla or Manali are good summer destinations in India (general knowledge from training data). An *agent* can go further — look up the cheapest flight from Delhi to Shimla on a specific date by calling a live API, and even complete the booking if asked.

How an Agent Works: Reasoning + Tools

An agent has two capabilities a plain chatbot lacks: **reasoning** (it can break a query down step by step, using techniques such as *Chain-of-Thought prompting*) and **tool access** (it can call external functions or APIs to get real data or perform real actions).

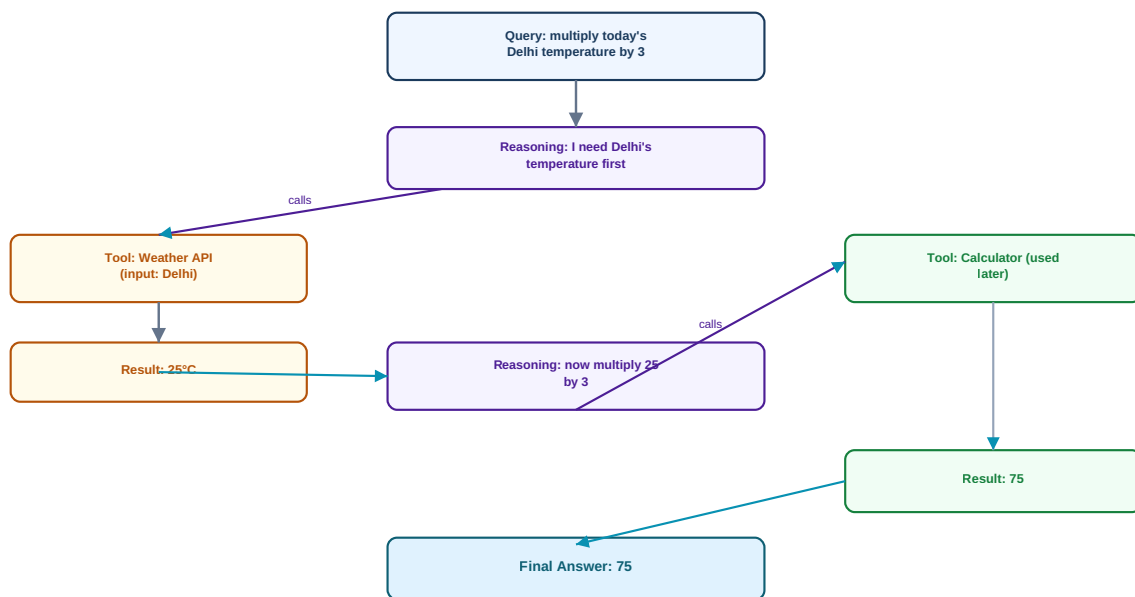


Figure 2.4 — An agent given a Weather API tool and a Calculator tool, reasoning step by step to answer "multiply today's Delhi temperature by 3."

Example

Given the query “Can you multiply today's temperature of Delhi with 3?”, and access to two tools — a calculator and a weather API — the agent reasons: first it needs Delhi's current temperature, so it calls the weather API and gets, say, 25°C. It then reasons that it needs to multiply 25 by 3, so it calls the calculator tool with those inputs, gets 75, and returns 75 as the final answer.

INTERVIEW TIP

The cleanest way to define an agent in an interview: “an agent is a chatbot with reasoning capability and tool access, so it can take actions instead of only replying.” Follow up by naming Chain-of-Thought prompting as one reasoning technique.

Quick Revision — Agents

- An agent = a chatbot + reasoning capability + access to tools.
- Reasoning lets it break a query into steps (e.g., Chain-of-Thought prompting).
- Tools let it fetch real data (APIs) or perform real actions (bookings, calculations).
- Chatbots talk; agents can also act.

K. Key Takeaways

- LangChain is organized around six components:
 - together they cover the majority of the framework's functionality.
- `LLM` standardize access to any LLM provider, solving the problem of inconsistent native APIs; LangChain supports both language models (text → text) and embedding models (text → vector).
- `PromptTemplate` are highly sensitive inputs to an LLM; LangChain supports dynamic/reusable templates, role-based prompts, and few-shot prompting.
- `Chain` build pipelines where one stage's output automatically becomes the next stage's input — sequential, parallel, or conditional.
- `RetrievalQA` connect an application to external knowledge via four sub-components: Document Loader, Text Splitter, Vector Store, and Retriever.
- `Memory` solves the statelessness of LLM API calls, using strategies such as buffer memory, buffer window memory, summarizer-based memory, or custom memory.
- `Agents` are chatbots with reasoning ability and tool access, allowing them to take real-world actions rather than just reply.

R. Revision Sheet

The six components

Models, Prompts, Chains, Indexes, Memory, Agents.

Models, in one line

A standardized interface for talking to any LLM or embedding model provider.

Prompt styles

Dynamic/reusable (placeholders), role-based (system + user), and few-shot (worked examples).

Chain types

Sequential (one after another), parallel (simultaneous, then combined), conditional (branching).

Indexes pipeline

Document Loader → Text Splitter → Vector Store → Retriever.

Memory types

Buffer memory, buffer window memory, summarizer-based memory, custom memory.

Agent, in one line

A chatbot with reasoning capability and tool access, able to take actions.

I. Interview Questions

Q1. Name the six core components of LangChain and describe each in one sentence.

- Q2.** What specific problem does the Models component solve, given that LLM APIs already exist?
- Q3.** What is the difference between a language model and an embedding model in LangChain?
- Q4.** Why are prompts described as highly "sensitive," and what does that imply for application design?
- Q5.** Explain few-shot prompting with an example use case.
- Q6.** What problem do chains solve, and what are the three chain patterns discussed in this chapter?
- Q7.** List and describe the four sub-components of Indexes.
- Q8.** Why are LLM API calls described as "stateless," and how does LangChain's Memory component address this?
- Q9.** Compare Conversation Buffer Memory and Conversation Buffer Window Memory.
- Q10.** What two capabilities distinguish an AI agent from a standard chatbot?

F. Further Reading

- [LangChain documentation — Chat Models section](#) (provider list & supported features)
- [LangChain documentation — Embedding Models section](#)
- [LangChain documentation — Prompt Templates & Few-Shot Prompting guides](#)
- [LangChain documentation — Memory module](#)
- [Chain-of-Thought Prompting](#) (research background for agent reasoning)